# Introduction to git and GitHub

Data Science Bootcamp

NASHVILLE SOFTWARE SCHOOL

# Objectives

1. Learn basic terminal commands for navigating your filesystem.
2. Become familiar with the git version control system.
3. Learn the difference between git and GitHub.
4. Learn the commands for cloning a repository and for adding, committing, and pushing work to GitHub.

# What is Git?

- A free and open source distributed **version control system**.
  - Keeps track of changes to a file, or set of files, over time so that you can recall specific versions later.
- Git organizes projects into **repositories** - directories containing the files that git is tracking.
  - Think of a repository like a folder on your computer- it keeps files for a project organized and in one place.

# What is GitHub?

- https://github.com/
- Hosts git repositories on the web.
- The repositories on GitHub  can be linked to a folder on your computer so that changes can be reflected in both locations.

# Cloning a git Repository

- When you accept a class project assignment, a repository specific to that project is created for you on GitHub.
- We'll make a copy of this repository on your computer by **cloning** this repository.
- This is done by using Git Bash (for those using Windows) or Terminal (for those using Mac).
  - These are applications that allow you to enter commands on your computer to connect with and perform operations with your Git repository.

# Command Line

The command line allows you to interact with your computer through a series of text commands.

We'll look at a few to get you started, but as you get more comfortable working with the command line, there are many more that you can learn.

Additional Resources:

- https://learnpythonthehardway.org/python3/appendixa.html
- https://www.datacamp.com/courses/introduction-to-shell-for-data-science

# Command Line

**pwd**

“print working directory” - tells you where you are located

**ls**

lists the contents of the current directory

**cd <directory name>**

changes the directory to the one specified

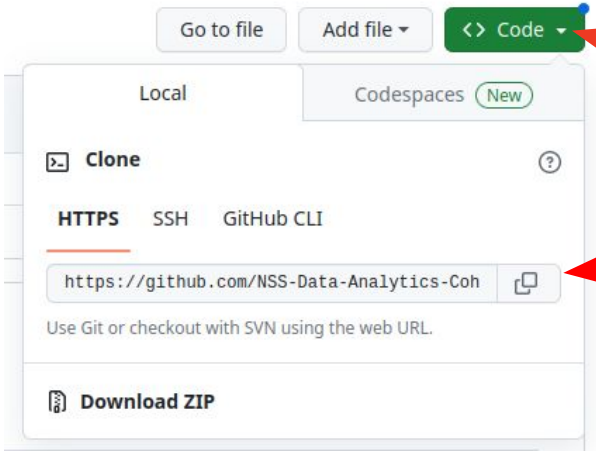
**cd ..**

takes you up on level in the directory structure

**mkdir <directory name>**

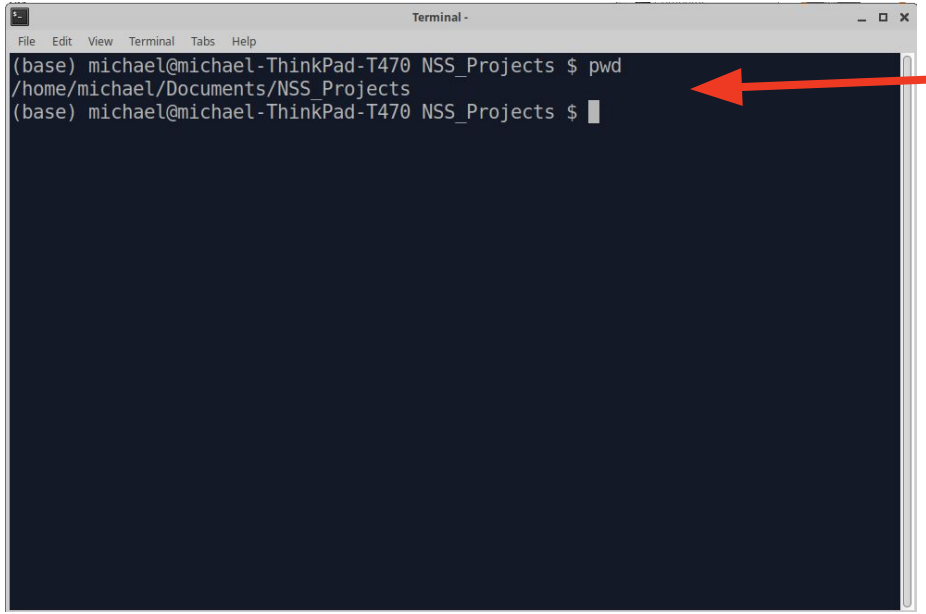
creates a new directory with the specified name

# Cloning a git Repository




1. Navigate in your browser to your repository. In the upper right click on the **Code** button.
2. Then click this button to copy the url.
3. Navigate to the **projects** folder for the relevant unit in your file structure

# Cloning a git Repository




4. Navigate in your terminal window to the directory where you would like your repository to live.
5. If you have the correct file path, then type:

```
git clone url_you_copied
```

Note: you must right click and select paste to paste the copied link. Ctrl+v (and other such keyboard shortcuts) will not work in Git Bash

# Creating a Personal Access Token

Before we can sync our local and remote repositories, we must create a personal access token on GitHub. Note: you only need to do this process once.

1. Navigate to github.com and make sure that you are logged into your account.
2. Open settings by clicking on your profile picture in the upper right corner.
3. In the menu on the left, scroll to the bottom and choose **Developer Settings**, then choose **Personal access tokens** -> **Tokens (classic)**
4. Choose **Generate new token** -> **Generate new token (classic)**, give it a name, and ensure that you have selected the repo scope.
5. Take the resulting token and save it in a file somewhere (or email to yourself).

# Pushing Changes to GitHub

After completing a unit of work **and** at the end of class, you should push your updates to GitHub. This process syncs your remote repository (on GitHub) with the changes to the file that you saved on your computer.

1. Save all changes to files you've worked on.
2. Navigate to your repository folder within your projects folder in your file structure.

# Pushing Changes to GitHub

Once the Git Bash window is open, type the following commands:

- **`git status`** (then press enter)
  - This shows a list of all changes made to files in the folder.
- **`git add name_of_file`** (then press enter)
  - You can also add multiple files at the same time by listing them or by using this command multiple times. Typing a period instead of a file name will add all files in the folder. Do this with caution. When we get to the point of using large files of data, we will need to be very careful NEVER to add/commit/push those files.
- **`git commit -m "meaningful message"`** (then press enter)
  - Example a of useful commit message: "Completed question 3"
  - The first time you do this, you'll likely need to configure your name and email address.
- **`git push origin main`** (then press enter)
  - The first time you do this, you'll need to input the PAT you generated.

# Additional Resources

We will be using git, GitHub, and Git Bash throughout the course, and we will go into details about additional features along the way. What we've covered so far is plenty to get you started!

This is one of very few areas of the course where it's best **not** to experiment too much without guidance, and please talk with your instructors if there is any sign of something going awry. Things can get tangled up and hard to undo quickly!

If you are interested, here are some resources to learn more:
https://docs.github.com/en/get-started/using-git/about-git
https://www.geeksforgeeks.org/working-on-git-bash/